Faculty of Technology and Engineering

Chandubhai S. Patel Institute of Technology (CSPIT)

Department of Computer Science & Engineering

Date: / /

Laboratory Manual

Academic Year	:	2024-25	Semester	:	4
Course code	:	CSE206	Course name	:	DATABASE MANAGEMENT SYSTEM

Practical - 9
Aim: You are a database administrator for a company managing sales, customers, and orders. The system must provide robust querying capabilities to retrieve, analyze, and manipulate data based on specific business requirements. Your task is to use SQL JOIN commands and constraints to answer real-world queries about salespeople, customers, and orders. (Refer the attached excel sheet for Database) 1. Salespeople: Contains information about sales staff, including their ID, name, city, and commission rate. 2. Customer: Stores details of customers, their ratings, and the salesperson managing them. 3. Order: Tracks order details, including the order amount, date, and links to customer and salesperson records.

C23CS23.sql | Welcome Page | C23CS43.sql

SQL Worksheet | History

0.161 seconds

Worksheet | Query Builder

```
--PRACTICAL 9


-- Create Sales People Table with unique name
-- Create Salesperson Table
CREATE TABLE SALES_PEOPLE2 (
    SALESPERSON_ID INT PRIMARY KEY,          -- Unique salesperson ID
    SNAME VARCHAR(100),                      -- Salesperson name
    CITY VARCHAR(100),                       -- City
    COMMISION_RATE DECIMAL(5, 2)             -- Commission rate
);

-- Create Customer Table
CREATE TABLE CUSTOMER2 (
    CUSTOMER2_ID INT PRIMARY KEY,            -- Unique identifier for the customer
    CUSTOMER2_NAME VARCHAR(100) NOT NULL,    -- Customer name, can't be null
    CITY VARCHAR(100),                       -- City (Nullable)
    RATING DECIMAL(3, 2) DEFAULT 5.00,       -- Customer rating (Default: 5.00)
    SALESPERSON2_ID INT,                     -- Foreign key for Salesperson managing the customer
    FOREIGN KEY (SALESPERSON2_ID) REFERENCES SALES_PEOPLE2(SALESPERSON_ID)  -- Link to Salesperson table
);

-- Create Order Table
```

Script Output

Task completed in 0.161 seconds

```
Error report -
ORA-02449: unique/primary keys in table referenced by foreign keys
02449. 00000 -  "unique/primary keys in table referenced by foreign keys"
*Cause:    An attempt was made to drop a table with unique or
           primary keys referenced by foreign keys in another table.
*Action:   Before performing the above operations the table, drop the
           foreign key constraints in other tables. You can see what
           constraints are referencing a table by issuing the following
           command:
           SELECT * FROM USER_CONSTRAINTS WHERE TABLE_NAME = "tabnam";

Table CUSTOMER2 dropped.


Table SALES_PEOPLE2 dropped.


Table SALES_PEOPLE2 created.


Table CUSTOMER2 created.
```

C23CS23.sql | Welcome Page | C23CS43.sql

SQL Worksheet | History

0.086 seconds

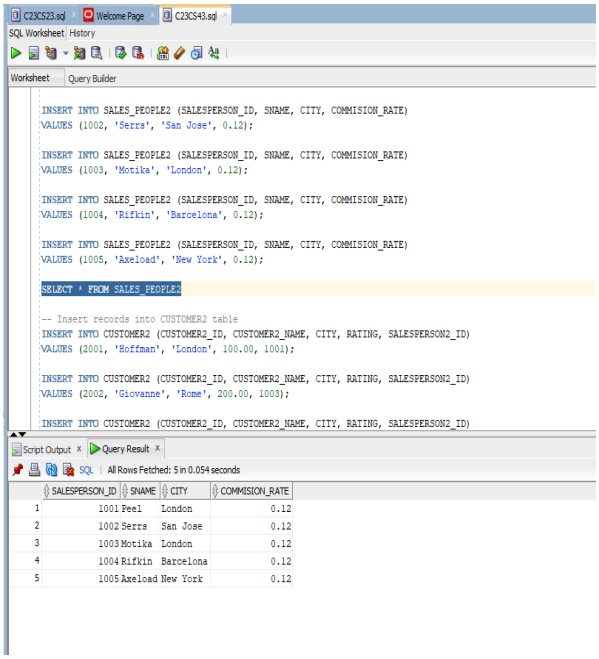
Worksheet | Query Builder

```
INSERT INTO Sales_order (Order_no, Order_date, Client_no, Dely_addr, Salesman_no, Dely_type, Order_status)
VALUES ('O010', TO_DATE('2025-02-10', 'YYYY-MM-DD'), 'C010', '1010 Green Way, Miami, FL 33101', 'S010', 'F', 'Fulfilled');


SELECT * FROM SALES_ORDER




--PRACTICAL 9


-- Create Sales People Table with unique name
-- Create Salesperson Table
CREATE TABLE SALES_PEOPLE2 (
    SALESPERSON_ID INT PRIMARY KEY,          -- Unique salesperson ID
    SNAME VARCHAR(100),                      -- Salesperson name
    CITY VARCHAR(100),                       -- City
    COMMISION_RATE DECIMAL(5, 2)             -- Commission rate
);

-- Create Customer Table
CREATE TABLE CUSTOMER2 (
```

Script Output

Task completed in 0.086 seconds

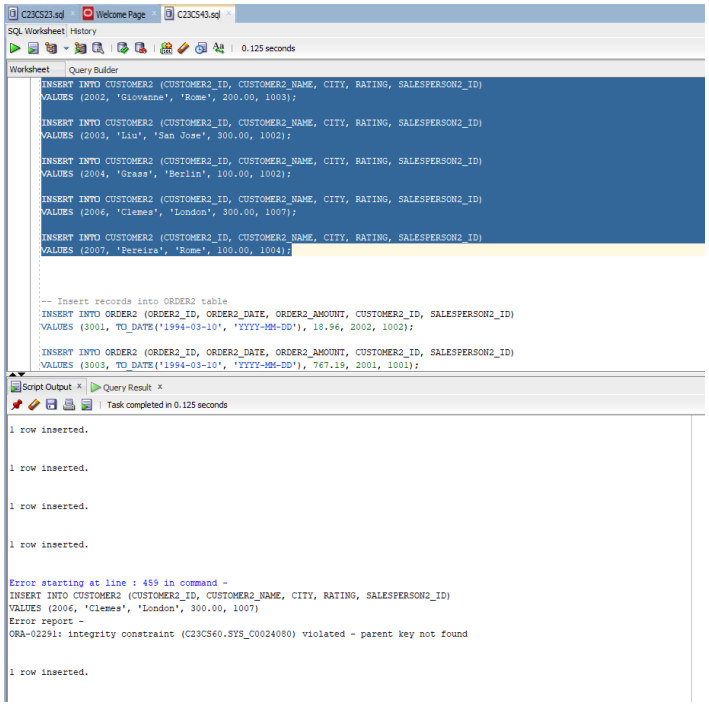
```
Error starting at line : 572 in command -
DROP TABLE SALES_PEOPLE2
Error report -
ORA-02449: unique/primary keys in table referenced by foreign keys
02449. 00000 -  "unique/primary keys in table referenced by foreign keys"
*Cause:    An attempt was made to drop a table with unique or
           primary keys referenced by foreign keys in another table.
*Action:   Before performing the above operations the table, drop the
           foreign key constraints in other tables. You can see what
           constraints are referencing a table by issuing the following
           command:
           SELECT * FROM USER_CONSTRAINTS WHERE TABLE_NAME = "tabnam";

Table CUSTOMER2 dropped.


Table SALES_PEOPLE2 dropped.


Table SALES_PEOPLE2 created.
```



```
INSERT INTO SALES_PEOPLE2 (SALESPERSON_ID, SNAME, CITY, COMMISION_RATE)
VALUES (1002, 'Serrs', 'San Jose', 0.12);

INSERT INTO SALES_PEOPLE2 (SALESPERSON_ID, SNAME, CITY, COMMISION_RATE)
VALUES (1003, 'Motika', 'London', 0.12);

INSERT INTO SALES_PEOPLE2 (SALESPERSON_ID, SNAME, CITY, COMMISION_RATE)
VALUES (1004, 'Rifkin', 'Barcelona', 0.12);

INSERT INTO SALES_PEOPLE2 (SALESPERSON_ID, SNAME, CITY, COMMISION_RATE)
VALUES (1005, 'Axeload', 'New York', 0.12);

SELECT * FROM SALES_PEOPLE2

-- Insert records into CUSTOMER2 table
INSERT INTO CUSTOMER2 (CUSTOMER2_ID, CUSTOMER2_NAME, CITY, RATING, SALESPERSON2_ID)
VALUES (2001, 'Hoffman', 'London', 100.00, 1001);

INSERT INTO CUSTOMER2 (CUSTOMER2_ID, CUSTOMER2_NAME, CITY, RATING, SALESPERSON2_ID)
VALUES (2002, 'Giovanne', 'Rome', 200.00, 1003);

INSERT INTO CUSTOMER2 (CUSTOMER2_ID, CUSTOMER2_NAME, CITY, RATING, SALESPERSON2_ID)
```

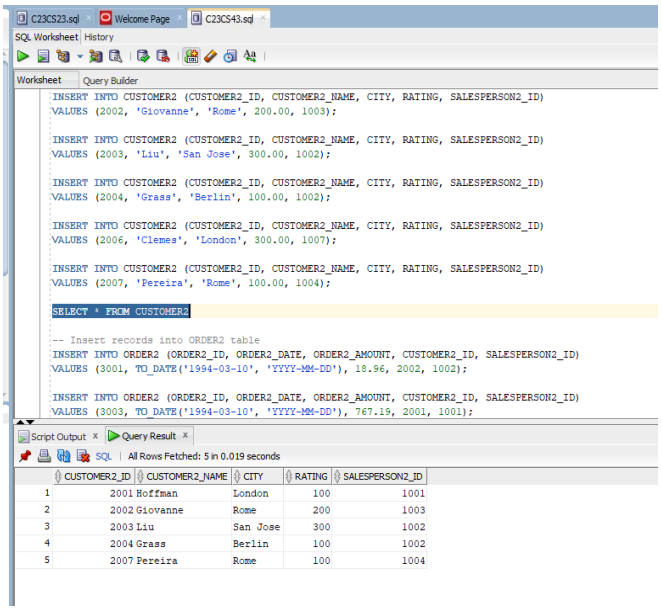
All Rows Fetched: 5 in 0.054 seconds

	SALESPERSON_ID	SNAME	CITY	COMMISION_RATE
1	1001	Peel	London	0.12
2	1002	Serrs	San Jose	0.12
3	1003	Motika	London	0.12
4	1004	Rifkin	Barcelona	0.12
5	1005	Axeload	New York	0.12



```
INSERT INTO CUSTOMER2 (CUSTOMER2_ID, CUSTOMER2_NAME, CITY, RATING, SALESPERSON2_ID)
VALUES (2002, 'Giovanne', 'Rome', 200.00, 1003);

INSERT INTO CUSTOMER2 (CUSTOMER2_ID, CUSTOMER2_NAME, CITY, RATING, SALESPERSON2_ID)
VALUES (2003, 'Liu', 'San Jose', 300.00, 1002);

INSERT INTO CUSTOMER2 (CUSTOMER2_ID, CUSTOMER2_NAME, CITY, RATING, SALESPERSON2_ID)
VALUES (2004, 'Grass', 'Berlin', 100.00, 1002);

INSERT INTO CUSTOMER2 (CUSTOMER2_ID, CUSTOMER2_NAME, CITY, RATING, SALESPERSON2_ID)
VALUES (2006, 'Clemes', 'London', 300.00, 1007);

INSERT INTO CUSTOMER2 (CUSTOMER2_ID, CUSTOMER2_NAME, CITY, RATING, SALESPERSON2_ID)
VALUES (2007, 'Pereira', 'Rome', 100.00, 1004);


-- Insert records into ORDER2 table
INSERT INTO ORDER2 (ORDER2_ID, ORDER2_DATE, ORDER2_AMOUNT, CUSTOMER2_ID, SALESPERSON2_ID)
VALUES (3001, TO_DATE('1994-03-10', 'YYYY-MM-DD'), 18.96, 2002, 1002);

INSERT INTO ORDER2 (ORDER2_ID, ORDER2_DATE, ORDER2_AMOUNT, CUSTOMER2_ID, SALESPERSON2_ID)
VALUES (3003, TO_DATE('1994-03-10', 'YYYY-MM-DD'), 767.19, 2001, 1001);
```

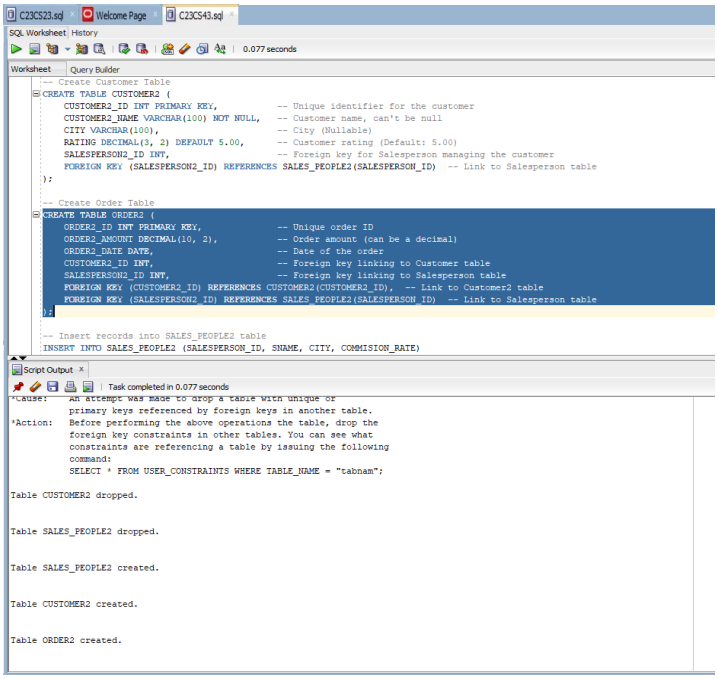
```
1 row inserted.


1 row inserted.


1 row inserted.


1 row inserted.


Error starting at line : 459 in command -
INSERT INTO CUSTOMER2 (CUSTOMER2_ID, CUSTOMER2_NAME, CITY, RATING, SALESPERSON2_ID)
VALUES (2006, 'Clemes', 'London', 300.00, 1007)
Error report -
ORA-02291: integrity constraint (C23CS60.SYS_C0024080) violated - parent key not found


1 row inserted.
```



```
INSERT INTO CUSTOMER2 (CUSTOMER2_ID, CUSTOMER2_NAME, CITY, RATING, SALESPERSON2_ID)
VALUES (2002, 'Giovanne', 'Rome', 200.00, 1003);

INSERT INTO CUSTOMER2 (CUSTOMER2_ID, CUSTOMER2_NAME, CITY, RATING, SALESPERSON2_ID)
VALUES (2003, 'Liu', 'San Jose', 300.00, 1002);

INSERT INTO CUSTOMER2 (CUSTOMER2_ID, CUSTOMER2_NAME, CITY, RATING, SALESPERSON2_ID)
VALUES (2004, 'Grass', 'Berlin', 100.00, 1002);

INSERT INTO CUSTOMER2 (CUSTOMER2_ID, CUSTOMER2_NAME, CITY, RATING, SALESPERSON2_ID)
VALUES (2006, 'Clemes', 'London', 300.00, 1007);

INSERT INTO CUSTOMER2 (CUSTOMER2_ID, CUSTOMER2_NAME, CITY, RATING, SALESPERSON2_ID)
VALUES (2007, 'Pereira', 'Rome', 100.00, 1004);

SELECT * FROM CUSTOMER2

-- Insert records into ORDER2 table
INSERT INTO ORDER2 (ORDER2_ID, ORDER2_DATE, ORDER2_AMOUNT, CUSTOMER2_ID, SALESPERSON2_ID)
VALUES (3001, TO_DATE('1994-03-10', 'YYYY-MM-DD'), 18.96, 2002, 1002);

INSERT INTO ORDER2 (ORDER2_ID, ORDER2_DATE, ORDER2_AMOUNT, CUSTOMER2_ID, SALESPERSON2_ID)
VALUES (3003, TO_DATE('1994-03-10', 'YYYY-MM-DD'), 767.19, 2001, 1001);
```

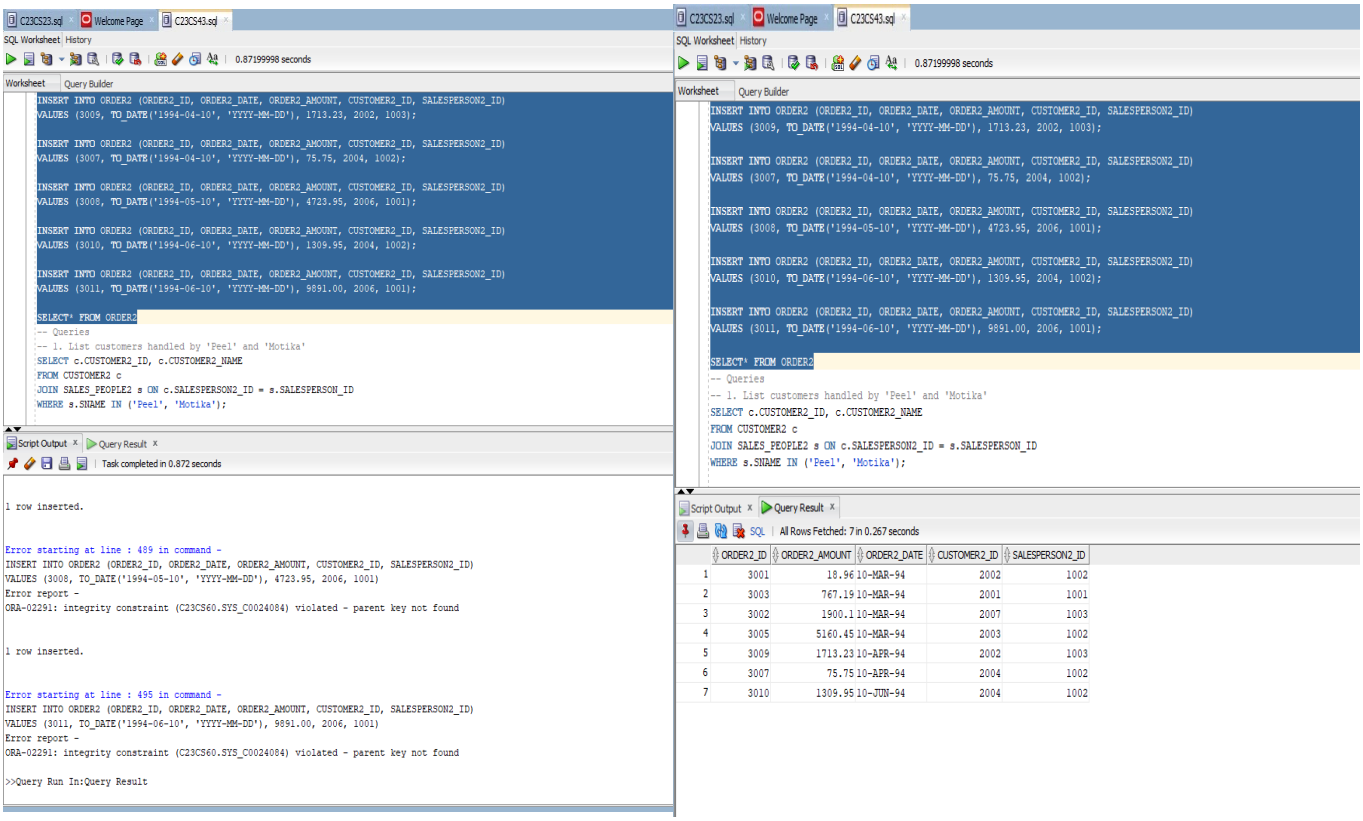
All Rows Fetched: 5 in 0.019 seconds

	CUSTOMER2_ID	CUSTOMER2_NAME	CITY	RATING	SALESPERSON2_ID
1	2001	Hoffman	London	100	1001
2	2002	Giovanne	Rome	200	1003
3	2003	Liu	San Jose	300	1002
4	2004	Grass	Berlin	100	1002
5	2007	Pereira	Rome	100	1004



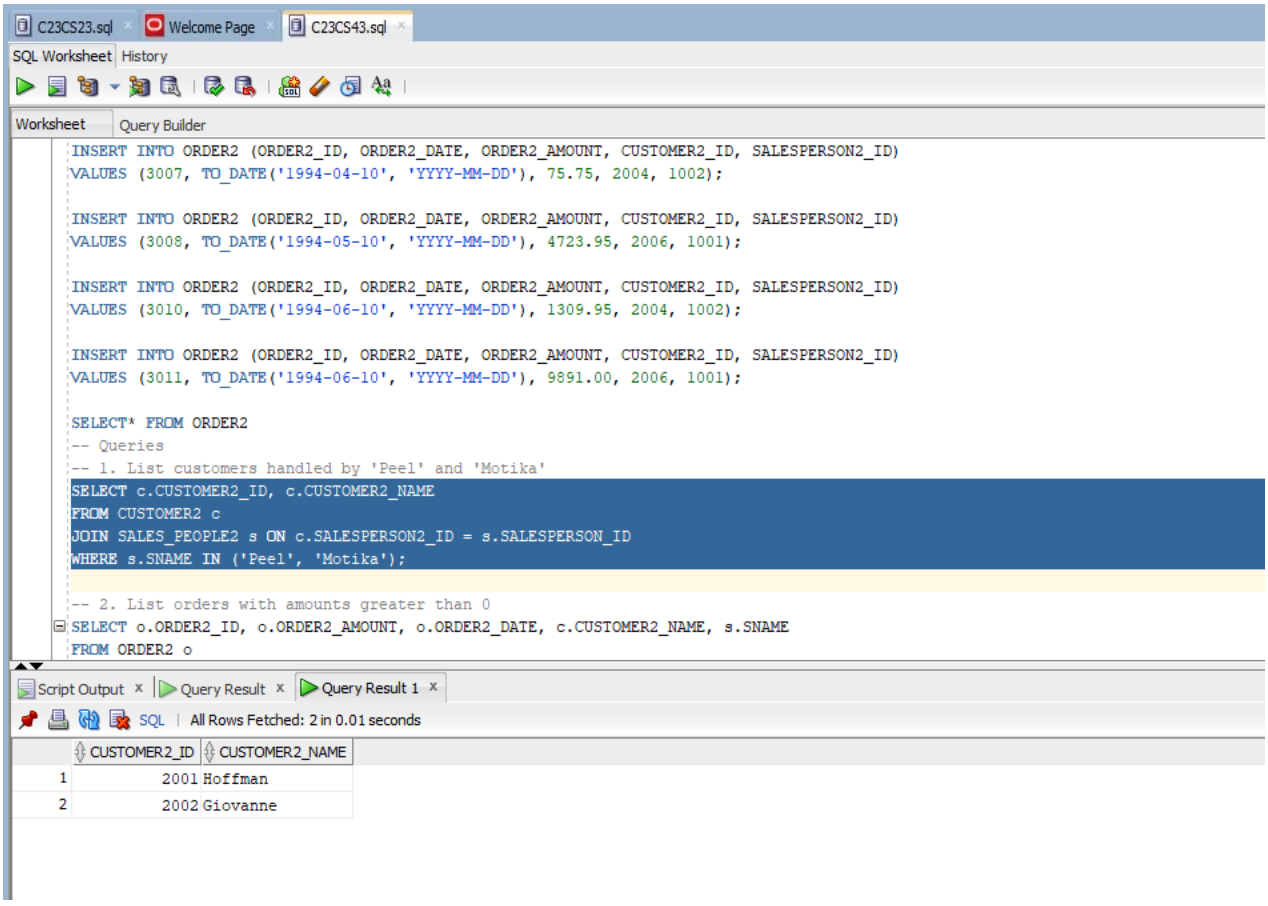
```
-- Create Customer Table
CREATE TABLE CUSTOMER2 (
    CUSTOMER2_ID INT PRIMARY KEY,           -- Unique identifier for the customer
    CUSTOMER2_NAME VARCHAR(100) NOT NULL,   -- Customer name, can't be null
    CITY VARCHAR(100),                      -- City (Nullable)
    RATING DECIMAL(3, 2) DEFAULT 5.00,      -- Customer rating (Default: 5.00)
    SALESPERSON2_ID INT,                    -- Foreign key for Salesperson managing the customer
    FOREIGN KEY (SALESPERSON2_ID) REFERENCES SALES_PEOPLE2(SALESPERSON_ID)  -- Link to Salesperson table
);

-- Create Order Table
CREATE TABLE ORDER2 (
    ORDER2_ID INT PRIMARY KEY,              -- Unique order ID
    ORDER2_AMOUNT DECIMAL(10, 2),           -- Order amount (can be a decimal)
    ORDER2_DATE DATE,                       -- Date of the order
    CUSTOMER2_ID INT,                       -- Foreign key linking to Customer table
    SALESPERSON2_ID INT,                    -- Foreign key linking to Salesperson table
    FOREIGN KEY (CUSTOMER2_ID) REFERENCES CUSTOMER2(CUSTOMER2_ID),  -- Link to Customer2 table
    FOREIGN KEY (SALESPERSON2_ID) REFERENCES SALES_PEOPLE2(SALESPERSON_ID)  -- Link to Salesperson table
);

-- Insert records into SALES_PEOPLE2 table
INSERT INTO SALES_PEOPLE2 (SALESPERSON_ID, SNAME, CITY, COMMISION_RATE)
```

```
*Cause:    An attempt was made to drop a table with unique or
           primary keys referenced by foreign keys in another table.
*Action:   Before performing the above operations the table, drop the
           foreign key constraints in other tables. You can see what
           constraints are referencing a table by issuing the following
           command:
           SELECT * FROM USER_CONSTRAINTS WHERE TABLE_NAME = "tabnam";

Table CUSTOMER2 dropped.


Table SALES_PEOPLE2 dropped.


Table SALES_PEOPLE2 created.


Table CUSTOMER2 created.


Table ORDER2 created.
```

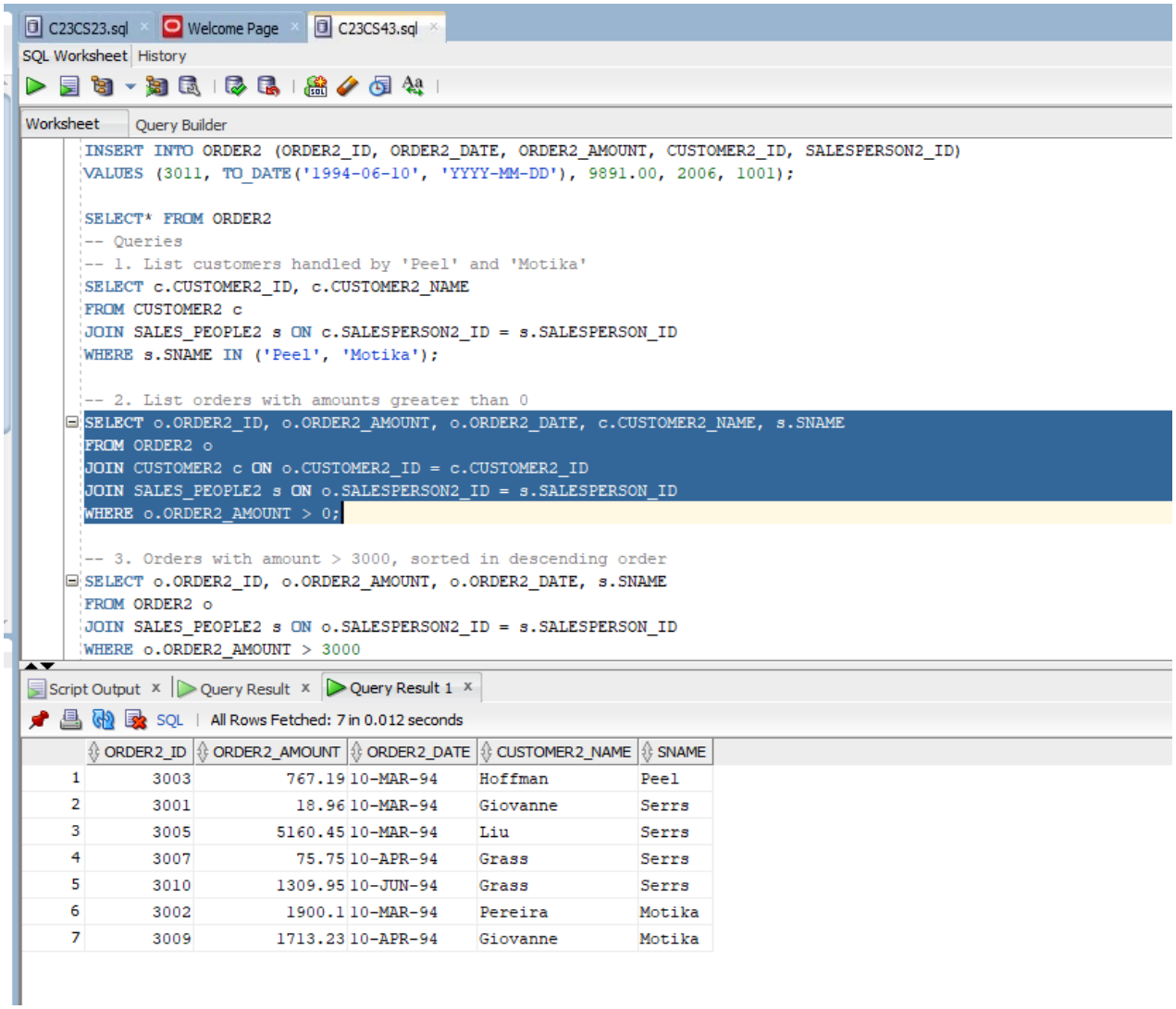


Tasks:-

1. The sales manager wants to identify all customers being handled by salespeople named Peel or Motika.

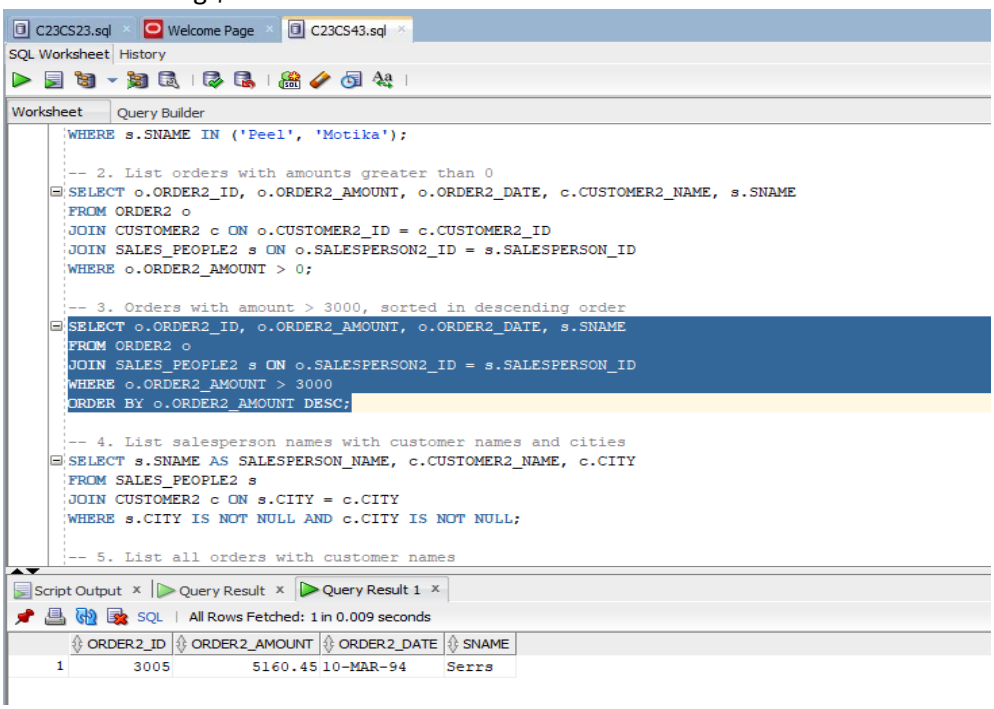


2. The finance team wants to exclude invalid orders where the amount is 0 or missing.



	ORDER2_ID	ORDER2_AMOUNT	ORDER2_DATE	CUSTOMER2_NAME	SNAME
1	3003	767.19	10-MAR-94	Hoffman	Peel
2	3001	18.96	10-MAR-94	Giovanne	Serrs
3	3005	5160.45	10-MAR-94	Liu	Serrs
4	3007	75.75	10-APR-94	Grass	Serrs
5	3010	1309.95	10-JUN-94	Grass	Serrs
6	3002	1900.1	10-MAR-94	Pereira	Motika
7	3009	1713.23	10-APR-94	Giovanne	Motika

3. The management wants to analyze the highest order values processed by salespeople for orders exceeding $3000.

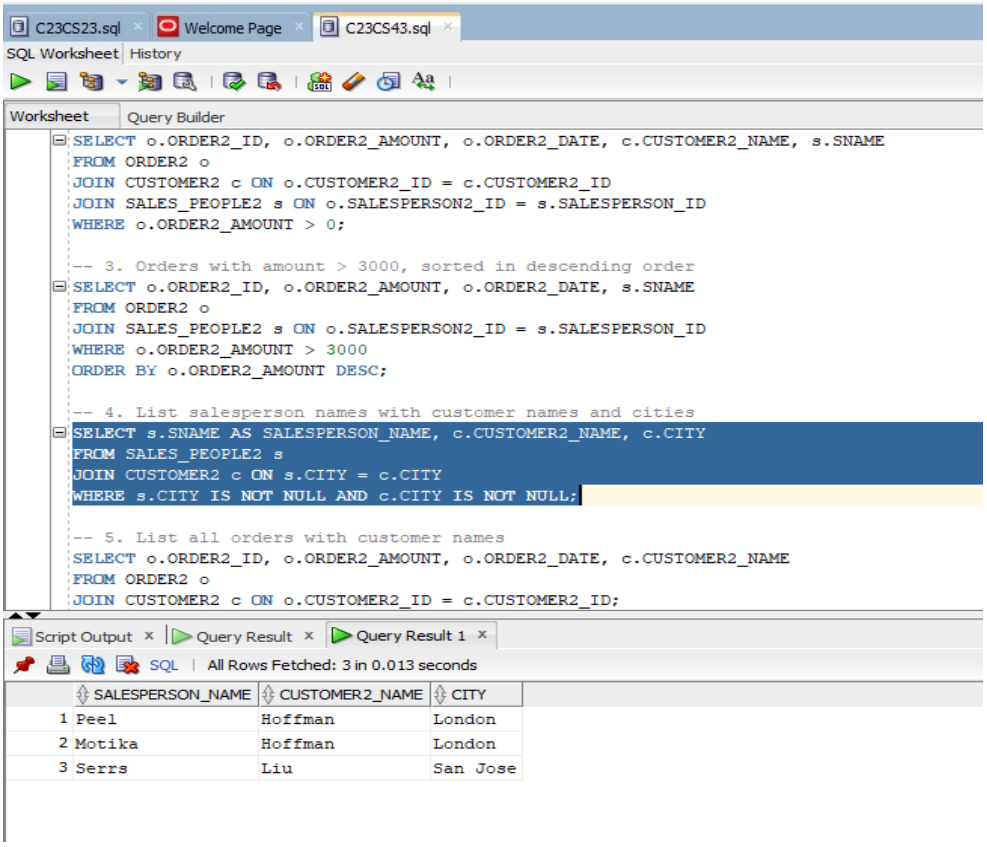


```
WHERE s.SNAME IN ('Peel', 'Motika');

-- 2. List orders with amounts greater than 0
SELECT o.ORDER2_ID, o.ORDER2_AMOUNT, o.ORDER2_DATE, c.CUSTOMER2_NAME, s.SNAME
FROM ORDER2 o
JOIN CUSTOMER2 c ON o.CUSTOMER2_ID = c.CUSTOMER2_ID
JOIN SALES_PEOPLE2 s ON o.SALESPERSON2_ID = s.SALESPERSON_ID
WHERE o.ORDER2_AMOUNT > 0;

-- 3. Orders with amount > 3000, sorted in descending order
SELECT o.ORDER2_ID, o.ORDER2_AMOUNT, o.ORDER2_DATE, s.SNAME
FROM ORDER2 o
JOIN SALES_PEOPLE2 s ON o.SALESPERSON2_ID = s.SALESPERSON_ID
WHERE o.ORDER2_AMOUNT > 3000
ORDER BY o.ORDER2_AMOUNT DESC;

-- 4. List salesperson names with customer names and cities
SELECT s.SNAME AS SALESPERSON_NAME, c.CUSTOMER2_NAME, c.CITY
FROM SALES_PEOPLE2 s
JOIN CUSTOMER2 c ON s.CITY = c.CITY
WHERE s.CITY IS NOT NULL AND c.CITY IS NOT NULL;

-- 5. List all orders with customer names
```

Script Output | Query Result | Query Result 1 — All Rows Fetched: 1 in 0.009 seconds

	ORDER2_ID	ORDER2_AMOUNT	ORDER2_DATE	SNAME
1	3005	5160.45	10-MAR-94	Serrs

4. The HR department wants to find relationships between salespeople and customers in the same city.

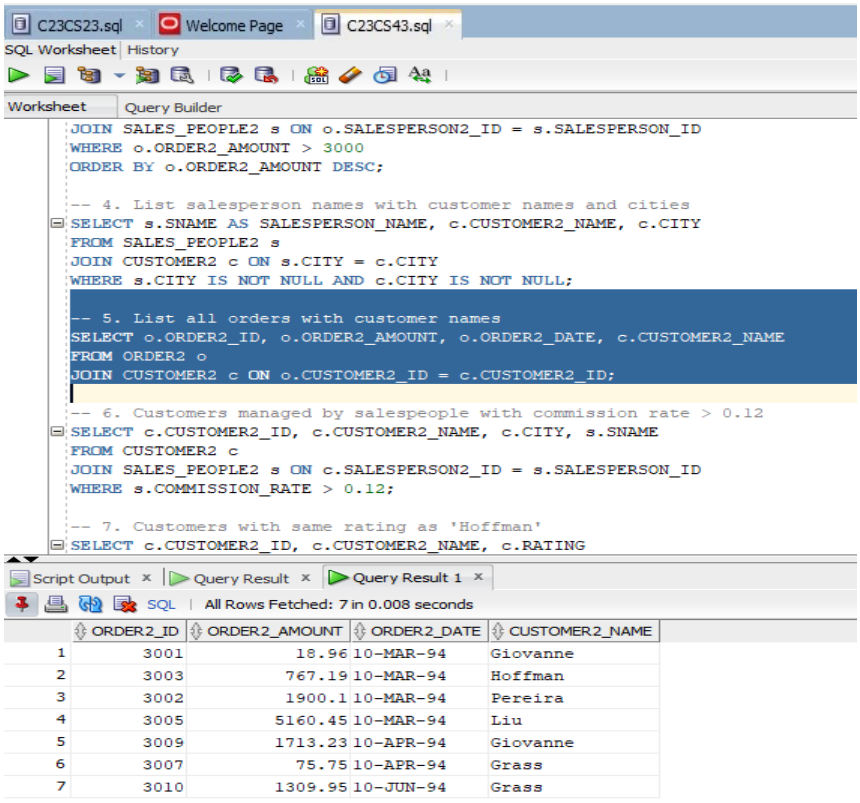


```
SELECT o.ORDER2_ID, o.ORDER2_AMOUNT, o.ORDER2_DATE, c.CUSTOMER2_NAME, s.SNAME
FROM ORDER2 o
JOIN CUSTOMER2 c ON o.CUSTOMER2_ID = c.CUSTOMER2_ID
JOIN SALES_PEOPLE2 s ON o.SALESPERSON2_ID = s.SALESPERSON_ID
WHERE o.ORDER2_AMOUNT > 0;

-- 3. Orders with amount > 3000, sorted in descending order
SELECT o.ORDER2_ID, o.ORDER2_AMOUNT, o.ORDER2_DATE, s.SNAME
FROM ORDER2 o
JOIN SALES_PEOPLE2 s ON o.SALESPERSON2_ID = s.SALESPERSON_ID
WHERE o.ORDER2_AMOUNT > 3000
ORDER BY o.ORDER2_AMOUNT DESC;

-- 4. List salesperson names with customer names and cities
SELECT s.SNAME AS SALESPERSON_NAME, c.CUSTOMER2_NAME, c.CITY
FROM SALES_PEOPLE2 s
JOIN CUSTOMER2 c ON s.CITY = c.CITY
WHERE s.CITY IS NOT NULL AND c.CITY IS NOT NULL;

-- 5. List all orders with customer names
SELECT o.ORDER2_ID, o.ORDER2_AMOUNT, o.ORDER2_DATE, c.CUSTOMER2_NAME
FROM ORDER2 o
JOIN CUSTOMER2 c ON o.CUSTOMER2_ID = c.CUSTOMER2_ID;
```

Script Output | Query Result | Query Result 1 — All Rows Fetched: 3 in 0.013 seconds

	SALESPERSON_NAME	CUSTOMER2_NAME	CITY
1	Peel	Hoffman	London
2	Motika	Hoffman	London
3	Serrs	Liu	San Jose

5. The operations team needs a list of orders along with the names of customers who placed them.

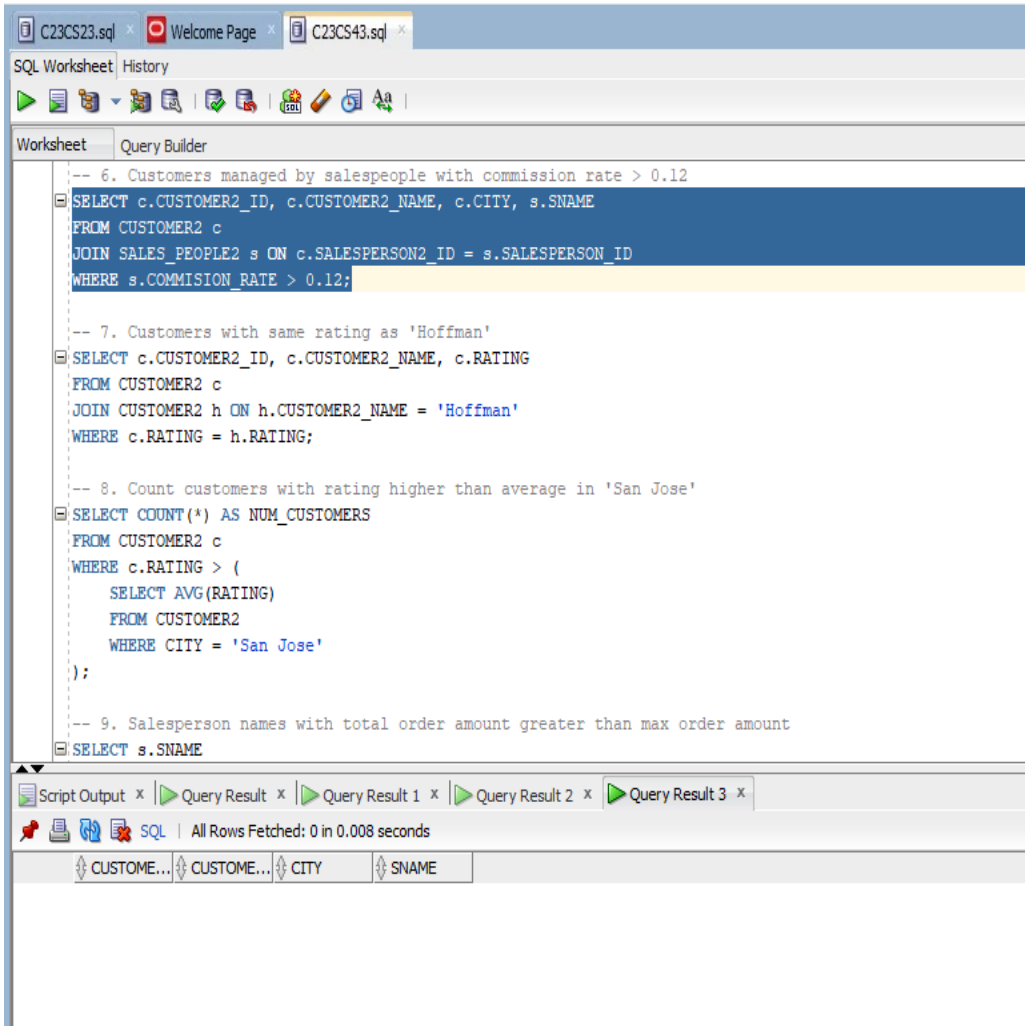


```
JOIN SALES_PEOPLE2 s ON o.SALESPERSON2_ID = s.SALESPERSON_ID
WHERE o.ORDER2_AMOUNT > 3000
ORDER BY o.ORDER2_AMOUNT DESC;

-- 4. List salesperson names with customer names and cities
SELECT s.SNAME AS SALESPERSON_NAME, c.CUSTOMER2_NAME, c.CITY
FROM SALES_PEOPLE2 s
JOIN CUSTOMER2 c ON s.CITY = c.CITY
WHERE s.CITY IS NOT NULL AND c.CITY IS NOT NULL;

-- 5. List all orders with customer names
SELECT o.ORDER2_ID, o.ORDER2_AMOUNT, o.ORDER2_DATE, c.CUSTOMER2_NAME
FROM ORDER2 o
JOIN CUSTOMER2 c ON o.CUSTOMER2_ID = c.CUSTOMER2_ID;

-- 6. Customers managed by salespeople with commission rate > 0.12
SELECT c.CUSTOMER2_ID, c.CUSTOMER2_NAME, c.CITY, s.SNAME
FROM CUSTOMER2 c
JOIN SALES_PEOPLE2 s ON c.SALESPERSON2_ID = s.SALESPERSON_ID
WHERE s.COMMISSION_RATE > 0.12;

-- 7. Customers with same rating as 'Hoffman'
SELECT c.CUSTOMER2_ID, c.CUSTOMER2_NAME, c.RATING
```

All Rows Fetched: 7 in 0.008 seconds

	ORDER2_ID	ORDER2_AMOUNT	ORDER2_DATE	CUSTOMER2_NAME
1	3001	18.96	10-MAR-94	Giovanne
2	3003	767.19	10-MAR-94	Hoffman
3	3002	1900.1	10-MAR-94	Pereira
4	3005	5160.45	10-MAR-94	Liu
5	3009	1713.23	10-APR-94	Giovanne
6	3007	75.75	10-APR-94	Grass
7	3010	1309.95	10-JUN-94	Grass

6. Identify customers managed by salespeople earning more than 12% commission

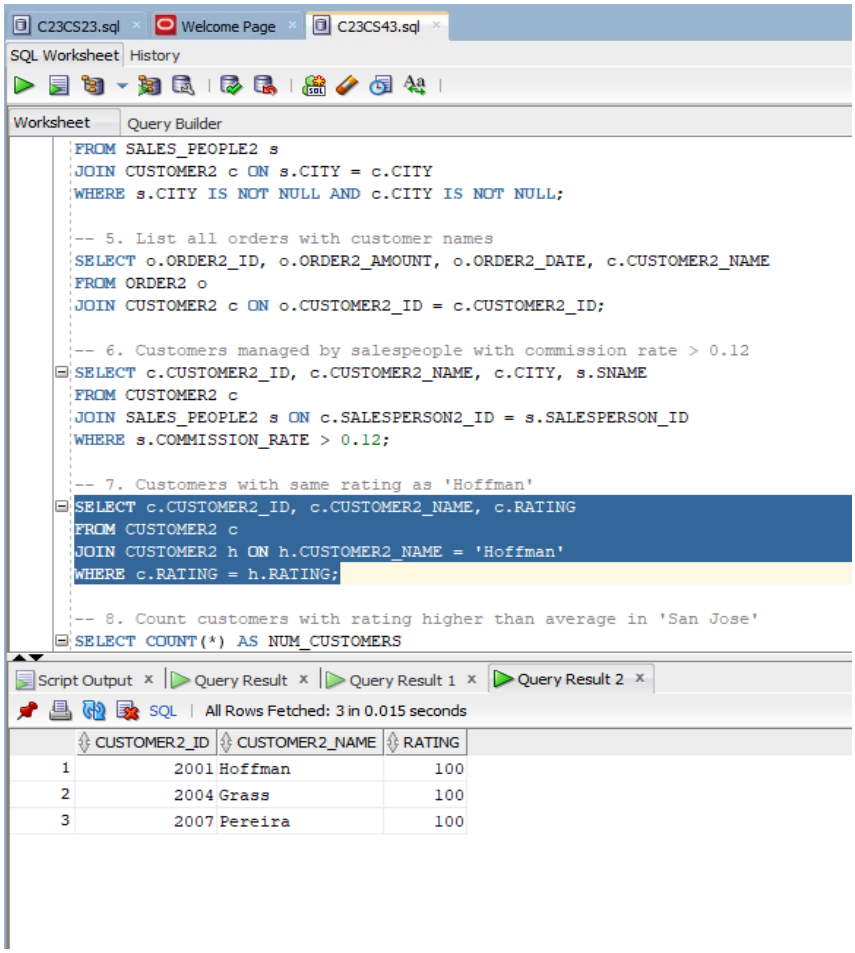


```
-- 6. Customers managed by salespeople with commission rate > 0.12
SELECT c.CUSTOMER2_ID, c.CUSTOMER2_NAME, c.CITY, s.SNAME
FROM CUSTOMER2 c
JOIN SALES_PEOPLE2 s ON c.SALESPERSON2_ID = s.SALESPERSON_ID
WHERE s.COMMISION_RATE > 0.12;

-- 7. Customers with same rating as 'Hoffman'
SELECT c.CUSTOMER2_ID, c.CUSTOMER2_NAME, c.RATING
FROM CUSTOMER2 c
JOIN CUSTOMER2 h ON h.CUSTOMER2_NAME = 'Hoffman'
WHERE c.RATING = h.RATING;

-- 8. Count customers with rating higher than average in 'San Jose'
SELECT COUNT(*) AS NUM_CUSTOMERS
FROM CUSTOMER2 c
WHERE c.RATING > (
    SELECT AVG(RATING)
    FROM CUSTOMER2
    WHERE CITY = 'San Jose'
);

-- 9. Salesperson names with total order amount greater than max order amount
SELECT s.SNAME
```

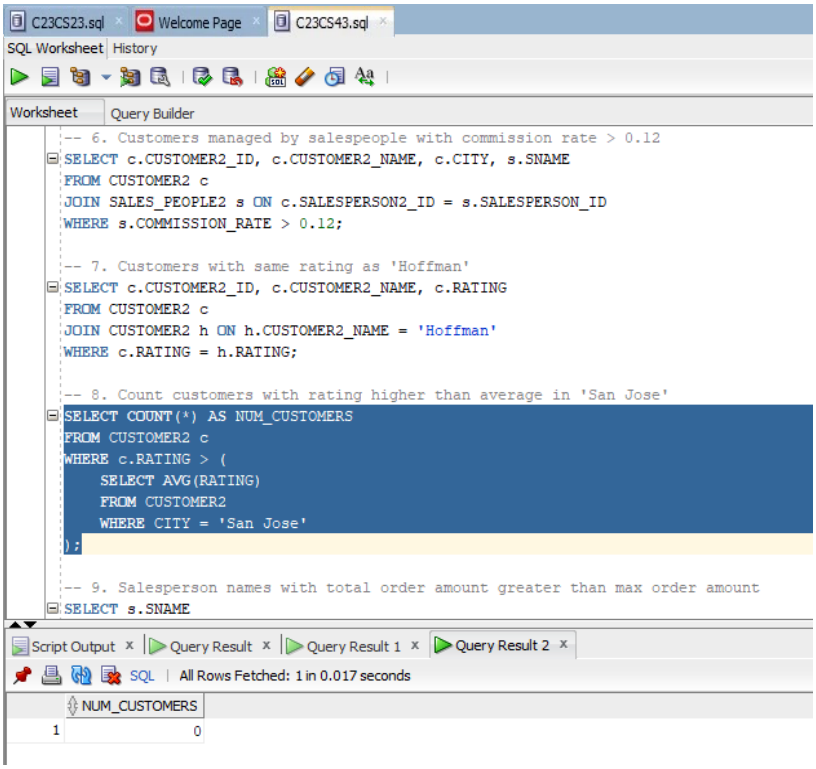
All Rows Fetched: 0 in 0.008 seconds

CUSTOME...	CUSTOME...	CITY	SNAME

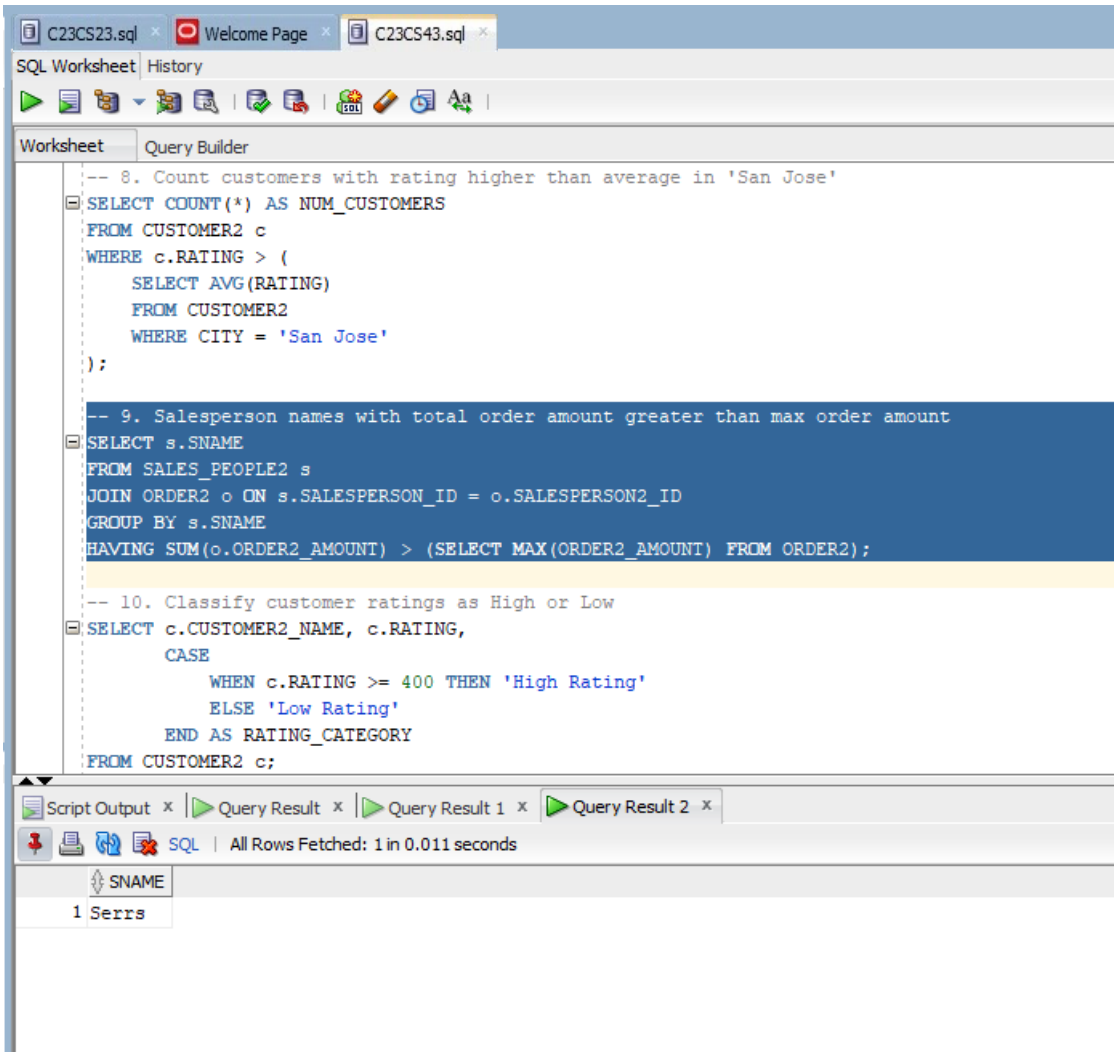
7. Identify customers with a rating identical to Hoffman's.



8. The analytics team wants to find the number of customers with ratings exceeding the average rating of customers from San Jose.



9. Identify salespeople whose total orders exceed the largest single order amount in the system.



10. Create a categorized listing of customers based on their ratings. Ratings >= 400 should be marked as 'High Rating'; otherwise, 'Low Rating'.